shieldify

Crush Trading

SECURITY REVIEW

Date: 14 May 2026

CONTENTS

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Crush Trading

Crush Trading is an AI-driven quantitative trading platform that uses automated agents to analyze market data and execute trading strategies. It processes large datasets such as price movements, news, and market signals to generate predictive insights and automatically place trades with minimal human intervention.

The goal is to improve trading efficiency by combining AI analysis, algorithmic strategies, and automated execution, enabling users to make faster, data-driven investment decisions.

Learn more: here

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 21 days with a total of 336 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying eight High, twelve Medium and eight Low severity issues. They're mainly related to insecure authentication and session management, improper secrets handling, vulnerable dependencies, broken swap and cross-chain logic, insufficient input validation, and multiple API-level security gaps, including injection risks, data leakage, and missing rate limiting.

The Crush team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	**Crush Trading**
Repository	crush-wallet-service
Type of Project	AI-assisted Quant Trading Platform
Security Review Timeline	21 days
Review Commit Hash	f5c376c5bd4e24ea27a2b18c69ee021663d4b170
Fixes Review Commit Hash	8122ad780181f47bc5ceef6c44c5a5e28ae71b00

5.2 Scope

The files from the following folders were in the scope of the security review:

Folders
src/config
src/constants
src/domain
src/handlers
src/infrastructure
src/middleware
src/presentation
src/schemas
src/service
src/shared
src/swagger
src/types
src/utils
src/main.ts

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **High** issues: 8
- **Medium** issues: 12
- **Low** issues: 8
- **Info** issues: 7

ID	Title	Severity	Status
[H-01]	Vulnerable JavaScript Dependency – Next.js 14.2.26 with Multiple Critical CVEs	High	Fixed
[H-02]	Weak Email OTP Authentication with Insufficient Entropy	High	Fixed
[H-03]	API Keys Stored in Plaintext in MongoDB	High	Fixed
[H-04]	Consider Verifying Excessive Session Persistence (No Idle Timeout / Extended Auth Lifetime Observed)	High	Fixed
[H-05]	Only First Step Executed in `executeSwap()`, Causing All EVM ERC20 Cross-Chain Swaps to Fail	High	Fixed
[H-06]	Incorrect Output Token Amount Extraction in `executeSwap()`	High	Fixed
[H-07]	WSOL Incorrectly Routed to Native SOL Transfer Path in `sendSolanaTransfer()`	High	Fixed
[H-08]	Wrapped SOL (WSOL) Balance Fetched Using Native SOL Method in `getSolanaTokenBalance()`	High	Fixed
[M-01]	NoSQL Regex Injection on News Trading Feed API	Medium	Fixed
[M-02]	Verbose Database Error Disclosure on News Trading Feed API	Medium	Fixed
[M-03]	Missing Input Validation on API Pagination Parameters	Medium	Fixed
[M-04]	Missing Rate Limiting on Referral Code Validation Endpoint	Medium	Fixed
[M-05]	Vercel WAF Browser Block Bypassed via Profile Switching	Medium	Fixed
[M-06]	Embedded Data Exfiltration in Auth Handler	Medium	Fixed
[M-07]	Consider Refactoring `getPrivyUserId()` Due to Possible IDOR on All Authenticated Endpoints	Medium	Fixed
[M-08]	Whitelist Schema Excludes Solana Addresses	Medium	Fixed
[M-09]	Debug Logging Leaks Sensitive Data in Production	Medium	Fixed
[M-10]	Input Validator Middleware Inconsistently Applied	Medium	Fixed
[M-11]	Price Field Optional for Non-Market Orders in `placeOrderWithPrivy()`	Medium	Fixed
[M-12]	`expiresAfter` Field Never Included in Request Body For Some Methods	Medium	Fixed
[L-01]	Sensitive User Data Exposure in Client-Side JWT Claims	Low	Fixed
[L-02]	Wildcard CORS Policy Exposes All Endpoints to Cross-Origin Attacks	Low	Fixed
[L-03]	IP-Spoofable Rate Limiting with In-Memory Store	Low	Fixed
[L-04]	Docker Container Runs as Root	Low	Fixed

[L-05]	Loss of Funds via Misconfigured Bridge Address in `depositUsdcFromEthereumToHyperLiquidWithPrivy()`	Low	Fixed
[L-06]	WSOL Balance Overwritten by SPL Token Scan in `getAllSolanaTokenBalances()`	Low	Fixed
[L-07]	`symbol` Passed as String Where Hyperliquid API Requires Numeric Coin Index in `placeOrderWithPrivy()`	Low	Fixed
[L-08]	USDC Balance Checked Using Native SOL Method for SPL Token in `depositFromSolanaWithFeeSponsorship()`	Low	Fixed
[I-01]	Consider Hardening Referrals Due To Farming via Unrestricted Account Creation	Info	Acknowledged
[I-02]	Misleading Error Message for Minimum Deposit Validation in `depositUsdcFromArbitrumToHyperLiquidWithPrivy()`	Info	Fixed
[I-03]	Redundant Branch in Secret Pattern Sanitizer	Info	Fixed
[I-04]	Duplicated Code Instances on `src/types/okx.ts`	Info	Fixed
[I-05]	`console.log()` Left in Production Code in Multiple Instances	Info	Fixed
[I-06]	Redundant `SOLANA_TOKEN_ADDRESS` Constant Duplicates `WRAPPED_SOL_ADDRESS`	Info	Fixed
[I-07]	Multiple token Accounts for the same mint can still overwrite balance	Info	Fixed

7. Findings

[H-01] Vulnerable JavaScript Dependency - Next.js 14.2.26 with Multiple Critical CVEs

Severity

High Risk

Description

The application frontend at www.crush.xyz uses Next.js version 14.2.26, identified through the JavaScript bundle served at `/_next/static/chunks/main-1ede66574bb5a7e4.js`. This version contains multiple known vulnerabilities with published CVEs, several of which are rated High to Critical severity. The vulnerabilities affect core Next.js functionality, including Image Optimization, Middleware, and the App Router, creating multiple attack vectors for information disclosure, SSRF, phishing, and denial of service.

Vulnerable Scenario: Execute a GET request to:

```
GET /_next/static/chunks/main-1ede66574bb5a7e4.js HTTP/2
Host: www.crush.xyz

Detected: Next.js version 14.2.26
```

CVE-2025-48068: Information exposure in Next.js dev server (lack of origin verification)
CVE-2025-57752: Image Optimization cache key confusion - cached responses served to
↪ unauthorized users
CVE-2025-55173: Image Optimization arbitrary file download via attacker-controlled external
↪ sources
CVE-2025-57822: Middleware SSRF via reflected sensitive headers in NextResponse.next()
CVE-2025-55184: App Router DoS via malicious HTTP request deserialization, causing CPU
↪ hang
CVE-2025-67779: Incomplete fix for CVE-2025-55184 - infinite loop in React Server
↪ Components

Affected Files

Frontend example: `/_next/static/chunks/main-1ede66574bb5a7e4.js`

Impact

Several impact scenarios according to CVEs.

Recommendation

Immediately upgrade Next.js to the latest 14.x patched version (>= 14.2.32) to address all known CVEs.

Team Response

Fixed.

[H-02] Weak Email OTP Authentication with Insufficient Entropy

Severity

High Risk

Description

The application's sole authentication mechanism is a 6-digit numeric One-Time Password (OTP) sent via email through Privy.io. This provides only 1,000,000 possible combinations (~19.93 bits of entropy), which is critically insufficient for an application managing cryptocurrency wallets and funds.

The OTP code (observed value: 428995) consists of digits 0-9 only, with no alphanumeric or special character complexity. There is no secondary authentication factor: the email OTP is the only barrier between an attacker and full account access, including control over the user's Ethereum and Solana wallets.

Combined with the lack of rate limiting observed on other endpoints (CWS-007), if the OTP verification endpoint also lacks rate limiting, an attacker could brute-force the entire keyspace in under 17 minutes at 1,000 requests per second. Even at a conservative 10 requests per second, the full space is exhausted in approximately 28 hours.

Vulnerable Scenario:

```
/ Authentication flow observed:
// 1. User enters email Privy sends 6-digit OTP to email
// 2. User enters OTP Full account access granted
// 3. No additional factor (no password, no TOTP, no hardware key)

// OTP entropy analysis:
// Character set: 0-9 (10 characters)
// Length: 6 digits
// Keyspace: 10^6 = 1,000,000 combinations
// Entropy: log2(1,000,000) = 19.93 bits

// Brute-force estimates (if no rate limiting on OTP endpoint):
// At 1,000 req/s: ~16.7 minutes
// At 100 req/s: ~2.8 hours
// At 10 req/s: ~27.8 hours

// Privy confirmation dialog shows:
// 'Please check legion.of.the.twilight@gmail.com
// for an email from privy.io and enter your code below.'
// [6 single-digit input boxes]
```

Affected Files

Frontend Sign-in mechanism

Impact

Account takeover: An attacker who knows or guesses a target's email address can initiate the login flow and brute-force the OTP to gain full access to the victim's account, including their cryptocurrency wallets.

Recommendation

- Implement rate limiting on the OTP verification endpoint: maximum 5 attempts per OTP, with account lockout after consecutive failures.
- Increase OTP entropy: use 8+ character alphanumeric codes (36^8 = ~41 bits) or implement TOTP with 8-digit codes.

Team Response

Fixed.

[H-03] API Keys Stored in Plaintext in MongoDB

Severity

High Risk

Description

The `ApiKeyModel` in `src/models/api-keys.ts` stores API keys as raw strings in MongoDB and performs lookups via direct string comparison. The unique index is also on the raw value.

Vulnerable Scenario:

Consider the following scenario:

```
# Scenario: Attacker gains read access to MongoDB (via Finding #8, SSRF, backup leak, etc.)
mongosh "mongodb://target-host:27017/wallets"

# Dump all API keys —immediately usable, zero cracking
db.apiKeys.find({}, { apiKey: 1, type: 1 })
# Output:
# { apiKey: "sk_live_a1b2c3d4e5f6...", type: "TokenRadar" }
# { apiKey: "sk_live_x9y8z7w6v5u4...", type: "ATS" }
```

Each key can now be used with the IDOR to drain any user's wallet:

```
curl -X POST https://api.crush.wallet/api/v1/send-transfer \
  -H "x-api-key: sk_live_a1b2c3d4e5f6..." \
  -d '{"privyUserId":"did:privy:ANY_USER","chain":"ethereum",...}
```

Affected Files

`src/models/api-keys.ts`

Impact

- Any MongoDB access (backup, SSRF, misconfiguration, insider) exposes all API keys in immediately usable form.

Recommendations

- Store a one-way hash (SHA-256 or bcrypt) of each API key instead of plaintext
- Compare hashes at lookup time. The in-memory cache in `auth.ts` already caches validated results, so the performance impact of hashing per cache miss is negligible.

Team Response

Fixed.

[H-04] Consider Verifying Excessive Session Persistence (No Idle Timeout / Extended Auth Lifetime Observed)

Severity

High Risk

Description

The application maintains an authenticated session for an excessive duration. During testing, after logging in and leaving the site unused for **>24 hours**, revisiting the application still showed the account as **logged in**, without requiring re-authentication.

For a platform that handles **money, trading, deposits/withdrawals, and sensitive account actions**, long-lived sessions without a strict **idle timeout** and/or **maximum session lifetime** significantly increase the window for account compromise (e.g., shared devices, stolen cookies, unattended browsers, endpoint compromise).

Vulnerable Scenario: The following steps help understand the issue: 1. Log in to the platform with a valid user account. 2. Close the tab/browser (or leave it idle) and do not use the application for **24+ hours**. 3. Return to the site and observe that the user is still authenticated (session remains valid) and can access account functionality without re-authentication.

Affected Files

Frontend logout handler (token/session revocation + cookie clearing)

Impact

If an attacker gains access to a user's device/browser profile or session tokens (e.g., via malware, shared workstation, unlocked device, stolen cookies, or unattended session), they may be able to perform unauthorized actions within a **large time window**, potentially resulting in **account takeover and asset loss**.

Recommendations

- Enforce **idle session timeout** (e.g., 10–30 minutes of inactivity) for all authenticated sessions.
- Enforce a **maximum session lifetime** (absolute timeout), even with activity (e.g., 8–24 hours depending on risk appetite).
- Ensure **logout** performs **server-side session invalidation** (revocation) in addition to client-side cookie/token clearing.

Team Response

Fixed.

[H-05] Only First Step Executed in `executeSwap()`, Causing All EVM ERC20 Cross-Chain Swaps to Fail

Severity

High Risk

Description

The function `executeSwap()` retrieves the Relay quote and processes only `steps[0]`, executing a single transaction item before returning.

```
const step = quoteResponse.steps[0];
if (!step.items || !Array.isArray(step.items) || step.items.length === 0) {
   throw new Error('No transaction items in quote response');
}

const txItem = step.items[0];
```

According to Relay's documentation, the quote endpoint returns a `steps` array that represents a sequential set of actions the application must complete.

> "The quote endpoint returns a steps array. Think of this as a recipe your application must follow. You need to iterate through these steps and prompt the user to sign or submit them."

For any EVM ERC20 token swap or bridge, Relay returns exactly two steps

```
"steps": [
  {
    "id": "approve",
    "action": "Confirm transaction in your wallet",
    "description": "Sign an approval for USDC",
    "kind": "transaction",
    "items": [ … ]
  },
  {
    "id": "deposit",
    "action": "Confirm transaction in your wallet",
    "description": "Depositing funds to the relayer to execute the swap",
    "kind": "transaction",
    "items": [ … ]
  }
]
```

The current code processes only `steps[0]`, the `approve` step and returns immediately after submitting it. The `deposit()` step in `steps[1]` is never reached. The function returns `success: true` despite the swap never having been executed.

For any empty steps, the docs specify the following
`Steps with no items, or an empty array of items, should be skipped`

Location of Affected Code

```
File: cross-chain-swap.service.ts
Function: executeSwap()
```

Note: This does not affect Solana deposits since Solana doesn't use ERC20-style approvals.

Proof Of Concept

The following gets a quote for swapping base USDC to hyperliquid. The response contains exactly two steps: approve and deposit

```
curl -X POST "https://api.relay.link/quote/v2" \
  -H "Content-Type: application/json" \
  -d '{
    "user": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
    "originChainId": 8453,
    "originCurrency": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
    "destinationChainId": 1337,
    "destinationCurrency": "0x0000000000000000000000000000000000000000",
    "recipient": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
    "tradeType": "EXACT_INPUT",
    "amount": "10000000"
```

“EXACT_INPUT”, “amount”: “10000000” \end{lstlisting}

```
"steps": [
    {
        "id": "approve",
        "action": "Confirm transaction in your wallet",
        "description": "Sign an approval for USDC",
        "kind": "transaction",
        "items": [
            {
                "status": "incomplete",
                "data": {
                    "from": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
                    "to": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
                    "data": "0x095ea7b3000000000000000000000000004cd...",
                    "value": "0",
                    "chainId": 8453,
                    "gas": "73112",
                    "maxFeePerGas": "6500000",
                    "maxPriorityFeePerGas": "1000000"
                }
            }
        ],
        "requestId": "0
↪ x4909d9f25dc4171620514946b231a3706d59328644fa1967e3f8f8733d1009cd"
    },

    {
        "id": "deposit",
        "action": "Confirm transaction in your wallet",
        "description": "Depositing funds to the relayer to execute the swap for USDC",
        "kind": "transaction",
        "items": [
            {
                "status": "incomplete",
                "data": {
                    "from": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
                    "to": "0x4cd00e387622c35bddb9b4c962c136462338bc31",
                    "data": "0
```

```
↪ xe80179520000000000000000000000003508bb71268bba25ecacc8f62...",
                    "value": "0",
                    "chainId": 8453,
                    "gas": "76442",
                    "maxFeePerGas": "6500000",
                    "maxPriorityFeePerGas": "1000000"
                },

            }
        ],
        "requestId": "0
↪ x4909d9f25dc4171620514946b231a3706d59328644fa1967e3f8f8733d1009cd",
        "depositAddress": ""
    }
]
```

Impact

If approval is needed for any tokens before swapping/bridging, the transaction would fail, as we would only execute one instruction. In the case above, the deposit step is never executed, meaning no swap would eventually happen, but the code still returns success.

Recommendation

Iterate over all steps and all items sequentially, executing each incomplete transaction item in order before proceeding to the next. This matches the execution model described in Relay's documentation:

```
if (!quoteResponse.steps || quoteResponse.steps.length === 0) {
  throw new Error('Invalid quote response: no steps returned');
}

const originChain = translateChainId(params.originChainId);
let lastResult: CrossChainSwapExecuteResult | null = null;

for (const step of quoteResponse.steps) {
  if (!step.items || step.items.length === 0) {
    logInfo(`Skipping step ${step.id}: no items`);
    continue;
  }

  for (const item of step.items) {
    if (item.status === 'complete') {
      logInfo(`Skipping completed item in step ${step.id}`);
      continue;
    }

    if (!item.data) {
      logInfo(`Skipping item with no transaction data in step ${step.id}`);
      continue;
    }
```

```
      logInfo(`Executing step: ${step.id}`);

      const outputTokenAmount = quoteResponse.details?.currencyOut?.amountFormatted;

      if (isEvmChain(originChain)) {
        lastResult = await this.executeEvmSwap(
          item.data,
          params.privyUserId,
          originChain,
          outputTokenAmount,
        );
      } else if (originChain === ChainId.Solana) {
        lastResult = await this.executeSolanaSwap(
          item.data,
          params.privyUserId,
          outputTokenAmount,
        );
      } else {
        throw new Error(`Unsupported origin chain: ${originChain}`);
      }
    }
  }

  if (!lastResult) {
    throw new Error('No executable steps found in quote response');
  }

  return lastResult;
```

References

- Relay API Quickstart — Steps

Team Response

Fixed.

[H-06] Incorrect Output Token Amount Extraction in `executeSwap()`

Severity

High Risk

Description

The function `executeSwap()` attempts to extract the output token amount from the Relay quote.

Response as follows:

```
const outputTokenAmount = txItem.data.outputTokenAmount ||
  quoteResponse.outputTokenAmount;
```

The `txItem.data.outputTokenAmount` and `quoteResponse.outputTokenAmount` do not exist in the Relay API response schema. According to the relay docs, the correct location of the output amount is

```
{
  "details": {
    "currencyOut": {
      "amount": "30754920",
      "amountFormatted": "30.75492",
      "amountUsd": "30.901612"
    }
  }
}
```

Both field paths accessed by the code are `undefined` in every valid Relay response, meaning `outputTokenAmount` will always be `undefined`. This value is then passed directly into `executeEvmSwap()` or `executeSolanaSwap()` as a parameter, propagating the incorrect value

This is the same bug in `getHyperliquidDepositQuote()`, where `relayQuote.outputTokenAmount` was also incorrectly accessed instead of `relayQuote.details.currencyOut.amountFormatted`.

The interface `RelayResponse` defined in `relay.ts` is misleading

Location of Affected Code

File: cross-chain-swap.service.ts
Function: executeSwap()

File: hyperliquid-cross-chain.ts
Function: getHyperliquidDepositQuote()

File: /src/types/relay.ts

Proof Of Concept

Call the quote api to see a sample response from the actual endpoint

```
curl -X POST "https://api.relay.link/quote/v2" \
  -H "Content-Type: application/json" \
  -d '{
    "user": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
    "originChainId": 8453,
    "originCurrency": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
    "destinationChainId": 1337,
    "destinationCurrency": "0x0000000000000000000000000000000000000000",
    "recipient": "0x03508bB71268BBA25ECaCC8F620e01866650532c",
    "tradeType": "EXACT_INPUT",
    "amount": "10000000"
  }' > response.json
```

OOOOOOO" }' > response.json \end{lstlisting}

Impact

`outputTokenAmount` will be `undefined` on every execution.

Recommendation

Extract the output amount from the correct path in the Relay response schema

```
const outputTokenAmount = quoteResponse.details.currencyOut.amountFormatted;
```

Modify the `RelayResponse` interface on `relay.ts` to accurately reflect the relay quote response.

Team Response

Fixed.

[H-07] WSOL Incorrectly Routed to Native SOL Transfer Path in `sendSolanaTransfer()`

Severity

High Risk

Description

The function `sendSolanaTransfer()` routes any transfer where `fromAsset` matches `SOLANA_TOKEN_ADDRESS` (the WSOL mint) to the native SOL transfer path:

```
const SOLANA_TOKEN_ADDRESS = 'So11111111111111111111111111111111111111112'; // WSOL

if (
  fromAsset.toUpperCase() === SOLANA_TOKEN_ADDRESS.toUpperCase() ||
  fromAsset === NATIVE_SOL_ADDRESS
) {
  return this.sendNativeSol(...);
}
```

However, WSOL is an SPL token and not native SOL. Since `sendNativeSol()` uses `SystemProgram.transfer()`, which transfers SOL, it cannot move WSOL balances. A user attempting to transfer WSOL will instead have native SOL debited from their wallet, or the transaction will fail on-chain if they have insufficient SOL.

Location of Affected Code

```
File: transfer.service.ts
Function: sendSolanaTransfer()
```

Impact

A user transferring WSOL will have one of two outcomes - **Wrong asset transferred**: If they have sufficient native SOL, that SOL is sent instead of their WSOL - **Transaction fails on-chain**: If they have insufficient native SOL, the `SystemProgram.transfer()` instruction fails, and the user receives a confusing error despite potentially having a large WSOL balance.

Recommendation

Remove WSOL from the native SOL branch. WSOL should go through the SPL token path (`sendSplToken()`), which correctly handles token accounts:

```
// Handle native SOL transfer
if (fromAsset === NATIVE_SOL_ADDRESS) {
  return this.sendNativeSol(
    privyUserId,
    chain,
    senderPublicKey,
    recipientPublicKey,
    amount,
    priorityRate,
  );
}

// Handle SPL token transfer
return this.sendSplToken(
  privyUserId,
  chain,
  senderPublicKey,
  recipientPublicKey,
  fromAsset,
  amount,
  priorityRate,
);
```

Team Response

Fixed.

[H-08] Wrapped SOL (WSOL) Balance Fetched Using Native SOL Method in `getSolanaTokenBalance()`

Severity

High Risk

Description

The function `getSolanaTokenBalance()` treats Wrapped SOL (`So11111111111111111111111111111111111111112`) identically to native SOL, using `connection.getBalance()` to fetch its balance.

```
if (
  tokenAddress === NATIVE_SOL_ADDRESS ||
  tokenAddress === WRAPPED_SOL_ADDRESS ||
  tokenAddress === SOLANA_TOKEN_ADDRESS
) {
  const balance = await connection.getBalance(publicKey);
  return balance / 1e9;
}
```

Wrapped SOL (WSOL) is **not** native SOL. It is an SPL token with mint address `So11111111111111111111111111111111111111112`, held in an associated token account. `connection.getBalance()` returns the native SOL lamport balance of the wallet, which is entirely unrelated to any WSOL held in the wallet's token accounts.

A user holding 100 WSOL and 0.01 SOL would have their balance reported as 0.01 rather than 100.

Location of Affected Code

```
File: balance.service.ts
Function: getSolanaTokenBalance()
```

Impact

Any feature that calls `getTokenBalance()` with a WSOL address will receive the wallet's native SOL balance instead of its WSOL balance.

The function `sendTransfer()`, which reads the balances and uses them to make transfer decisions, will always revert if the user attempts to transfer WSOL while they don't have an equal amount of SOL on their wallet.

Proof Of Concept

```
    try {
        balance = await balanceService.getTokenBalance(
          wallet.address,
          chain,
          fromAsset,
        );//@audit buggy see balance.ts : what happens if user has both sol/wsol
        // solana -> from -> wsol -> (bal = 5 wsol, 1 sol), returns 1 SOL
      }

      if (balance < amount) {
        throw new InsufficientBalanceError(balance, amount);
      }
```

Scenario User has 1 SOl and 5 WSOL User attempts to transfer WSOL, so `amount = 5 WSOL` and `getBalance()` returns SOL balance ie 1 SOL The check `if balance < amount` becomes `1 < 5` which will always revert with `InsufficientBalanceError`

Recommendation

Remove WSOL from the native SOL branch and handle it as a standard SPL token

```
// Only native SOL uses getBalance()
if (tokenAddress === NATIVE_SOL_ADDRESS) {
  const balance = await connection.getBalance(publicKey);
  return balance / 1e9;
}

// WSOL and all other SPL tokens fall through to the token account query below
```

WSOL will then be correctly handled by the existing SPL token path using `getParsedTokenAccountsByOwner()`, which reads the actual token account balance.

Team Response

Fixed.

[M-01] NoSQL Regex Injection on News Trading Feed API

Severity

Medium Risk

Description

The `/api/news-trading/feed` endpoint passes the query parameter directly into a MongoDB $regex operation without any sanitization or escaping. This was confirmed by submitting regex metacharacters (\codex{*, %00, (.+.+)+x}) and observing raw `MongoServerError` responses that reveal the internal query structure. The server response also echoes back the processed query and confirms the search is applied to the headline field via the searchBy parameter in the response body. An attacker can craft arbitrary regular expressions to enumerate data through blind regex matching, extract information from fields they should not have access to, and potentially cause resource exhaustion through complex regex patterns. The response also leaks internal query details, including the exact field being searched (searchBy: headline).

Vulnerable Scenario: Step 1: Confirm regex execution with a valid pattern:

```
GET /api/news-trading/feed?query=.*&page=1&limit=5
// Returns all results - regex .* matches everything
```

Step 2: Trigger error with an invalid regex metacharacter:

```
GET /api/news-trading/feed?page=1&limit=1&query=*
// Returns raw MongoServerError with internal trace IDs
```

Step 3: Confirm field disclosure via response echo:

```
GET /api/news-trading/feed?query=(.+.+)+x&page=1&limit=1
// Response includes: "searchQuery":{"query":"(. . ) x","searchBy":"headline"}
```

Step 4: Blind enumeration example:

```
GET /api/news-trading/feed?query=^Trump&page=1&limit=1
// Returns headlines starting with 'Trump' - enables character-by-character
↪ extraction
```

Affected Files

Frontend `/api/news-trading/feed`

Impact

- Information disclosure: The `searchQuery` echo in the response reveals the exact database field being queried and how the input is processed, providing attackers a roadmap for further exploitation.
- Potential ReDoS: While basic catastrophic backtracking patterns did not produce measurable delays in testing (likely due to MongoDB regex timeout configuration), more sophisticated patterns targeting known headline content could potentially cause CPU exhaustion on the database server.

Recommendations

- Sanitize all user input before passing to MongoDB $regex operations. At minimum, escape all regex metacharacters: . * + ? ^ $ { } [ ] ( ) |

- Implement a server-side allowlist of permitted search characters (alphanumeric, spaces, basic punctuation only).
- Remove the searchQuery echo from API responses to prevent information disclosure about internal query structure.
- Consider using MongoDB's $text index for full-text search instead of $regex, which provides built-in protection against injection.

Team Response

Fixed.

[M-02] Verbose Database Error Disclosure on News Trading Feed API

Severity

Medium Risk

Description

The `/api/news-trading/feed` endpoint returns raw MongoDB server error messages to the client, including internal error codes, BSON field names, trace IDs, and internal function arguments. These error messages provide attackers with detailed information about the database technology, query structure, and internal architecture, significantly reducing the effort required to craft targeted attacks. The errors are triggered by submitting malformed input in the query, page, and limit parameters, and are returned as plain JSON to unauthenticated callers.

Vulnerable Scenario:

```
// Pagination abuse triggers BSON error with internal details:
GET /api/news-trading/feed?page=0&limit=99999999999999&query=%27
{"code":3011500000,"message":"MongoServerError:BSON field 'skip'
value must be >= 0, actual value '-99999999999999'
[trace-id]333280e0-67ac-4362-9cfa-f1b2e75812d6",
"args":[],"traceId":"333280e0-67ac-4362-9cfa-f1b2e75812d6"}


// Regex error exposes internal trace IDs:
GET /api/news-trading/feed?page=1&limit=1&query=*
{"code":3011500000,"message":"MongoServerError:Regular expression is invalid:
quantifier does not follow a repeatable item
[trace-id]1a979340-9afd-4db3-adac-4b78df5e5da0",
"args":[],"traceId":"1a979340-9afd-4db3-adac-4b78df5e5da0"}
```

Affected Files

Frontend `/api/news-trading/feed?`

Impact

- Database technology fingerprinting: Error messages confirm MongoDB is the backend database, enabling targeted NoSQL injection techniques.
- Internal architecture mapping: Trace IDs, error codes, and BSON field names reveal internal query structure and processing pipeline.
- Attack surface expansion: Detailed errors guide attackers in crafting precision payloads (e.g., knowing the skip field is used for pagination tells them the exact MongoDB query pattern).
- This finding compounds NoSQL Regex Injection by providing attackers with immediate feedback on their injection attempts.

Recommendations

- Implement centralized error handling that returns generic error messages to clients (e.g., 'Invalid request parameters. Reference: ').
- Log detailed database errors server-side with correlation IDs for debugging, but never expose them to clients.
- Sanitize all error responses through middleware that strips MongoDB-specific error details, trace IDs, and BSON field information.

Team Response

Fixed.

[M-03] Missing Input Validation on API Pagination Parameters

Severity

Medium Risk

Description

The page and limit parameters on the `/api/news-trading/feed` endpoint accept arbitrary values without validation, including zero, negative numbers, and extremely large integers. These values are passed directly into MongoDB's `.skip()` and `.limit()` operations, causing server-side errors and potentially enabling data dumping through excessive limit values.

When `page=0` is combined with a large limit, the resulting negative skip value crashes the MongoDB query. When a very large limit is used with a valid page, the server may return the entire news collection in a single response, enabling bulk data extraction.

Vulnerable Scenario:

```
// page=0 with a large limit causes a negative skip calculation:
GET /api/news-trading/feed?page=0&limit=9999999999999&query=%27
// Formula: skip = (page - 1) * limit = (0 - 1) * 99999999999999
// Result: skip = -99999999999999 (MongoDB rejects negative skip)

// Large limit enables bulk data extraction:
GET /api/news-trading/feed?page=1&limit=99999&query=
// May return entire news collection in a single response
```

Affected Files

Frontend `api/news-trading/feed?page=1&limit=1&query=`

Impact

- Denial of service through repeated requests with malformed pagination that trigger database errors and consume server resources.
- Bulk data extraction by setting extremely high limit values to dump the entire news collection.
- Integer overflow potential if page * limit calculations exceed JavaScript's safe integer range, leading to unpredictable query behavior.
- Server resource exhaustion from MongoDB processing very large result sets.

Recommendations

- Implement strict server-side validation for all pagination parameters: page must be >= 1, limit must be between 1 and a reasonable maximum (e.g., 100).
- Use a validation library (Zod, Joi, class-validator) to enforce numeric types and bounds before any database operation.

Team Response

Fixed.

[M-04] Missing Rate Limiting on Referral Code Validation Endpoint

Severity

Medium Risk

Description

The POST `/api/referral/validate` endpoint allows unlimited rapid-fire submission of referral codes without any rate limiting, account lockout, CAPTCHA, or progressive delay mechanism. During testing, 5+ consecutive invalid referral code submissions were made in rapid succession with identical error responses and no throttling. While the referral code space itself is large (36^8 = ~2.8 trillion combinations, ~41 bits of entropy), the absence of rate limiting enables automated enumeration of valid referral codes. An attacker with sufficient bandwidth could discover valid codes to bypass the invite-only access control. The referral validation endpoint is hosted on www.crush.xyz (POST `/api/referral/validate`) and accepts a JSON body with the referralCode field. The endpoint is authenticated (requires Privy Bearer token) but performs no request-level throttling.

Vulnerable Scenario:

```
// Rapid-fire referral code submissions with no throttling:
POST /api/referral/validate
Host: www.crush.xyz
Authorization: Bearer eyJhbG...doJw
Content-Type: application/json

{"referralCode":"TC83HZRi"} "Failed to register with referral code"
{"referralCode":"TC83HZRA"} "Failed to register with referral code"
{"referralCode":"TC83HZRB"} "Failed to register with referral code"
{"referralCode":"AAAAAAAA"} "Failed to register with referral code"
{"referralCode":"00000000"} "Failed to register with referral code"
// All 5+ attempts: identical response, no delay, no lockout, no CAPTCHA
```

Affected Files

Frontend `/api/referral/validate`

Impact

- Referral code enumeration: Automated scripts can discover valid referral codes, bypassing the invite-only access control and enabling unauthorized account creation at scale.
- Invite system abuse: Valid referral codes could be sold or distributed, undermining the controlled growth strategy of the platform.
- Referral fraud: An attacker could discover and use other users' referral codes, potentially claiming referral bonuses or manipulating referral-based reward systems.

Recommendations

- Implement rate limiting on the /api/referral/validate endpoint: maximum 5 attempts per minute per user/IP.
- Add CAPTCHA or proof-of-work challenge after 3 consecutive failed attempts.

Team Response

Fixed.

[M-05] Vercel WAF Browser Block Bypassed via Profile Switching

Severity

Medium Risk

Description

The production frontend at `https://crush.xyz` is protected by Vercel's Security Checkpoint, which blocks suspicious browser sessions with a "Failed to verify your browser" page (Code 705). However, this block operates on a per-browser-profile (cookie/session) basis rather than at the IP level. An attacker who receives a block can immediately bypass it by opening a new browser profile, an incognito window, or simply clearing cookies and continuing browsing from the same IP address with no further challenge.

Vulnerable Scenario:

```
Browser Profile A visits crush.xyz BLOCKED (Code 705)
  "Failed to verify your browser"
  Vercel Security Checkpoint | cle1::1772125640-VmFn9iRHZzvnzwQDeG6XEabrBqUkNtID

Browser Profile B (same machine, same IP) visits crush.xyz ALLOWED
  Full site access, no challenge presented
```

Affected Files

Vercel WAF

Impact

The WAF provides a false sense of security against automated reconnaissance and attacks. Blocked sessions do not escalate to IP-level bans, allowing persistent attackers to continue from the same source. The protection is effective against casual/accidental abuse but not against targeted attacks.

Recommendations

- Escalate repeated WAF failures from the same IP to IP-level blocks (temporary ban after N failed browser verifications from the same source within a time window)
- Implement progressive challenge escalation: cookie-based → IP-based → CAPTCHA → hard block

Team Response

Fixed.

[M-06] Embedded Data Exfiltration in Auth Handler

Severity

Medium Risk

Description

The `handleCompleteMcpLogin()` function in `src/handlers/auth-handlers.ts` contains a `fetch()` call to a local HTTP endpoint (`http://127.0.0.1:7244/ingest/...`) that transmits sensitive JWT verification key metadata. This code is disguised with a `##region agent log` comment block and uses `.catch(()=>{})` to silently swallow failures, making it invisible during normal operation. It sends the key's length, whether it contains newlines, and whether it starts with `-----BEGIN` — information sufficient to fingerprint the key format and aid in key extraction or replay attacks.

Vulnerable Scenario: If a local service is running on port 7244 (or an attacker establishes a listener there), every MCP login completion will leak the JWT key metadata. In a containerized environment, `127.0.0.1` could be reached by sidecar containers in the same pod. The UUID in the path (`93e00504-...`) suggests a specific ingestion pipeline.

Affected Files

`src/handlers/auth-handlers.ts`

Impact

- Leaks cryptographic key metadata on every MCP login flow
- Could be an intentional backdoor left by a developer or supply chain compromise
- The silent `.catch(()=>{})` ensures it never surfaces in logs or error handlers

Recommendations

- Remove this entire code block.
- Conduct a code provenance review of recent commits, if possible.

Team Response

Fixed.

[M-07] Consider Refactoring `getPrivyUserId()` Due to Possible IDOR on All Authenticated Endpoints

Severity

Medium Risk

Description

The `getPrivyUserId()` function, used across every handler that performs wallet operations, accepts `privyUserId` from the request body/query when the caller authenticates with an API key. This means any client holding a valid API key (of type `TokenRadar` or `ATS`) can specify any user's `privyUserId` and execute operations on their behalf, including transfers, swaps, withdrawals, and trading operations

Vulnerable Scenario:

```
# With the current implementation of the function and considering there is no upper-layer
↪ protection
# An attacker with a valid API key can act as any user
curl -X POST https://target/api/v1/send-transfer \
  -H "x-api-key: VALID_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "privyUserId": "did:privy:VICTIM_USER_ID",
    "chain": "solana",
    "fromAsset": "So11111111111111111111111111111111111111112",
    "toWallet": "ATTACKER_WHITELISTED_ADDRESS",
    "amount": "100"
  }'
```

“ATTACKER_WHITELISTED_ADDRESS”, “amount”: “100” }’ \end{lstlisting}

Affected Files

`src/utils/handler-helpers.ts`

Impact

- Complete account takeover for any user whose `privyUserId` is known or guessed
- Unauthorized fund transfers, trading, leverage changes, and withdrawals

Recommendation

For API-key-authenticated requests, enforce that the `privyUserId` is bound to the API key at the `database/session` level, not supplied by the caller. Implement a mapping of API keys to authorized user scopes.

Team Response

Fixed.

[M-08] Whitelist Schema Excludes Solana Addresses

Severity

Medium Risk

Description

The whitelist address validation schema only accepts EVM-format addresses (`/^0x[a-fA-F0-9]{40}$/`), making it impossible to whitelist Solana addresses. Since the transfer handler enforces whitelist checks for all chains, including Solana, this effectively blocks all Solana transfers, or worse, if the whitelist check is bypassed for Solana in practice, it means Solana transfers have no whitelist protection.

Vulnerable Scenario:

```
// src/schemas/whitelist-schemas.ts
export const addWhitelistAddressSchema = z.object({
  privyUserId: z.string().startsWith('did:privy:').optional(),
  address: z.string().regex(/^0x[a-fA-F0-9]{40}$/), // Only EVM!
  label: z.string().min(1).max(50),
  chain: z.enum(['ethereum', 'arbitrum', 'base', 'solana', 'hyperliquid']),
});
```

The `chain` enum allows `solana`, but the `address` regex rejects Solana's base58 format. Meanwhile, in `withdrawal-whitelist.ts`, the `isWhitelisted()` function lowercases the address, which destroys Solana address integrity since base58 is case-sensitive.

Affected Files

`src/schemas/whitelist-schemas.ts`

Impact

- Users cannot whitelist Solana addresses through the API
- If Solana addresses are manually inserted into the database, the `toLowerCase()` in `isWhitelisted()` will corrupt them, causing legitimate whitelisted transfers to fail

Recommendation

Add chain-specific address validation. For Solana, validate base58 format (32-44 characters, base58 character set). Remove `toLowerCase()` for Solana addresses in the whitelist model or apply chain-specific normalization.

Team Response

Fixed.

[M-09] Debug Logging Leaks Sensitive Data in Production

Severity

Medium Risk

Description

Multiple locations contain console.log statements that leak sensitive information in production logs. Most critically, `src/middleware/auth.ts` logs the API key type result and allowed types on every authentication check, and `src/infrastructure/database/mongodb-client.ts` logs the full MongoDB connection URI (which typically contains credentials).

Vulnerable Scenario: CloudWatch/Datadog logs and sees:

```
INFO: Connected to MongoDB { uri: "mongodb://admin:SuperSecretP4ss@10.0.1.50:27017/
↪ crush-wallet" }
```

```
INFO: result TokenRadar
INFO: allowedTypes ["TokenRadar", "ATS"]
```

A compromised log aggregator, a developer with read-only log access, or a log export to a third-party SIEM now exposes database credentials and internal auth configuration. An attacker uses the MongoDB URI to connect directly.

Affected Files

`src/middleware/auth.ts`

Impact

- Log aggregation systems contain MongoDB credentials, API key metadata, and internal configuration details.

Recommendations

- Remove all `console.log()` debug statements from `auth.ts`
- Redact credentials from MongoDB URI before logging (only log the host/db name)
- Audit all `logInfo()`/`logError()` calls for sensitive data in their context objects

Team Response

Fixed.

[M-10] Input Validator Middleware Inconsistently Applied

Severity

Medium Risk

Description

The `inputValidatorMiddleware()` that detects prompt injection and wallet exploit patterns is only applied to two route groups: transfer handlers and Hyperliquid handlers. Other critical endpoints like OKX swap, cross-chain swap, whitelist management, and relay handlers do not have this protection.

Vulnerable Scenario:

```
// Transfer handlers —protected
const transferRoutes = new Hono()
  .use('*', authenticateApiKey([ApiKeyType.TokenRadar, ApiKeyType.ATS]))
  .use('*', inputValidatorMiddleware()) // Applied

// OKX handlers —NOT protected
const okxRoutes = new Hono()
  .use('*', authenticateApiKey([ApiKeyType.TokenRadar, ApiKeyType.ATS]))
  // No inputValidatorMiddleware()
```

```
// Cross-chain handlers —NOT protected
const crossChainRoutes = new Hono()
  .use('*', authenticateApiKey([ApiKeyType.TokenRadar, ApiKeyType.ATS]))
  // No inputValidatorMiddleware()
```

Affected Files

`/src/handlers/transfer-handlers.ts`

Impact

- Prompt injection attacks via OKX and cross-chain swap endpoints are not detected
- If the service is accessed by AI agents (as suggested by the MCP session support), unprotected endpoints are vulnerable to indirect prompt injection

Recommendation

Apply `inputValidatorMiddleware()` globally in `server.ts` before all route handlers, or apply it at the app level rather than per-route-group.

Team Response

Fixed.

[M-11] Price Field Optional for Non-Market Orders in `placeOrderWithPrivy()`

Severity

Medium Risk

Description

The function `placeOrderWithPrivy()` conditionally sets the price field `p` only when `params.price` is provided

```
if (params.price) {
  orderReq.p = params.price;
}
```

According to the Hyperliquid API documentation, the price field is **required** for limit and trigger orders. The current implementation silently omits the price for limit and trigger orders if the caller fails to provide it, resulting in a malformed request that will either be rejected by the API or execute with undefined price behaviour.

Location of Affected Code

```
File: hyperliquid.service.ts
Function: placeOrderWithPrivy()
```

Impact

A limit or trigger order submitted without a price field will either: - Be rejected by the Hyperliquid API with a validation error, failing silently from the caller's perspective, with no clear error message indicating the missing field - Execute at an unintended price if the API applies a default, leading to unexpected fills

Recommendation

Enforce price as required for limit and trigger orders.

Team Response

Fixed.

[M-12] `expiresAfter` Field Never Included in Request Body For Some Methods

Severity

Medium Risk

Description

The Hyperliquid API supports an optional `expiresAfter` field in the request body, which sets a timestamp after which the order will be rejected if not yet processed.

When placing an order using `placeOrderWithPrivy()`, the parameter `expiresAfter` determines how long the order remains active before it expires. However, the current implementation does not check if users have provided the value.

```
const requestBody = {
  action: orderAction,
  nonce,
  signature,
  vaultAddress: params.vaultAddress || null,
  // expiresAfter is never included
};
```

This allows the order to remain active even when the user wanted to limit how long the order would be active.

Location of Affected Code

```
File: hyperliquid.service.ts
Function: placeOrderWithPrivy()
Function: placeOrdersWithPrivy()
Function: cancelOrderWithPrivy()
```

Impact

Orders that are intended to expire after a specific time will instead be submitted with no expiry, remaining valid indefinitely. This is particularly dangerous for time-sensitive orders such as:

The caller believes the order will expire at the specified time. In reality, it will remain open until manually cancelled or filled, potentially executing at an unintended time under unfavourable market conditions.

Recommendation

Include `expiresAfter` in the request body when provided by the caller:

```
const requestBody: Record<string, unknown> = {
  action: orderAction,
  nonce,
  signature,
  vaultAddress: params.vaultAddress?.toLowerCase() || null,
  expiresAfter: params.expiresAfter,
};
```

The interfaces should also be updated to reflect that `expiresAfter` is an optional parameter.

Team Response

Fixed.

[L-01] Sensitive User Data Exposure in Client-Side JWT Claims

Severity

Low Risk

Description

The Privy ID token (privy-id-token cookie) contains sensitive user information in its JWT payload that is accessible client-side. Decoding the Base64-encoded JWT payload reveals the user's email address, Ethereum wallet address, Solana wallet address, wallet client types, chain types, account creation timestamp, Privy application ID, and user DID (Decentralized Identifier). While JWT-based authentication requires certain claims, the inclusion of linked wallet addresses and email in a client-accessible cookie significantly expands the information available to any XSS vulnerability or malicious browser extension. The JWT has a 10-hour lifetime (exp - iat = 36,000 seconds), providing a wide window for exploitation. The Privy application ID (cm15s036y0865nzzbgzg8fwnx) and user DID (did:privy:cmm3pmc5x01mz0djr2esfvapv) are also exposed, which could be used to target the Privy API directly or enumerate users.

```
// Decoded privy-id-token JWT payload:
// {
// "cr": "1772125252", // account creation timestamp
// "linked_accounts": "[
// {"type":"email","address":"legion.of.the.twilight@gmail.com","lv":1772204204},
// {"type":"wallet","address":"0xbcE1d4589171BA1092450588408Ce170586d2A6f",
```

```
// "chain_type":"ethereum","wallet_client_type":"privy","lv":1772125254},
// {"type":"wallet","address":"7mk7jkZb2ke7gnZT63aJHkwT8eUCNqvRSevcxZEjs1Hr",
// "chain_type":"solana","wallet_client_type":"privy","lv":1772125254}
// ]",
// "iss": "privy.io",
// "aud": "cm15s036y0865nzzbgzg8fwnx", // Privy app ID
// "sub": "did:privy:cmm3pmc5x01mz0djr2esfvapv", // user DID
// "exp": 1772240204 // 10-hour token lifetime
// }


// Token accessible via: document.cookie (privy-id-token cookie)
// or: Authorization header (Bearer token in API requests)
```

Affected Files

JWT token in HTTP requests

Recommendations

- Minimize JWT claims: Remove wallet addresses and email from the client-accessible ID token. Use server-side session lookups for this data instead.
- If wallet data must be in the JWT, encrypt the payload (JWE) so it cannot be decoded client-side.

Team Response

Fixed.

[L-02] Wildcard CORS Policy Exposes All Endpoints to Cross-Origin Attacks

Severity

Low Risk

Description

The server configures CORS with `origin: '*'`, allowing any website to make authenticated cross-origin requests to the API. Since authentication uses custom headers (`x-api-key`, `Authorization`), these are explicitly allowed in `allowHeaders`.

Affected Files

`src/presentation/server.ts`

Recommendation

Restrict `origin` to specific allowed domains (e.g., the Crush dashboard URL). For API-key-only endpoints, consider removing CORS entirely and serving them as backend-to-backend APIs.

Team Response

Fixed.

[L-03] IP-Spoofable Rate Limiting with In-Memory Store

Severity

Low Risk

Description

The MCP login rate limiter in `auth-handlers.ts` uses an in-memory `Map` and trusts the `cf-connecting-ip` or `x-forwarded-for` headers for client identification. Both headers are trivially spoofable without proper reverse proxy configuration. Additionally, the in-memory store does not persist across process restarts and does not work in multi-instance deployments (load-balanced environments).

Affected Files

`src/handlers/auth-handlers.ts`

Recommendation

Consider implementing rate limiting at the infrastructure level (this is partially handled by Vercel app already).

Team Response

Fixed.

[L-04] Docker Container Runs as Root

Severity

Low Risk

Description

The Dockerfile does not specify a non-root user. The Deno process runs as root inside the container, which increases the blast radius of any container escape or code execution vulnerability. Also note the broad Deno permissions: `--allow-net` (all network), `--allow-read` (all filesystem reads), `--allow-ffi` (foreign function interface), `--allow-sys` (system info).

Affected Files

Dockerfile configuration

Recommendation

Add `USER deno` or create a dedicated non-root user. Restrict Deno permissions to specific domains and paths (e.g., `--allow-net=api.relay.link,www.okx.com`).

Team Response

Fixed.

[L-05] Loss of Funds via Misconfigured Bridge Address in `depositUsdcFromEthereumToHyperLiquidWithPrivy()`

Severity

Low Risk

Description

The function `depositUsdcFromEthereumToHyperLiquidWithPrivy()` attempts to deposit usdc from ethereum to hyperliquid. The recipient address is defined as follows

```
const recipientAddress = receiverAddress ||
        ETHEREUM_USDC_CONFIG.BRIDGE_ADDRESS;
```

If `receiverAddress` is not provided, we default to `ETHEREUM_USDC_CONFIG.BRIDGE_ADDRESS` which is defined as follows

```
export const ETHEREUM_USDC_CONFIG = {
  ADDRESS: '0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48',
  DECIMALS: 6,
  DOMAIN_NAME: 'USD Coin',
  DOMAIN_VERSION: '2',
  BRIDGE_ADDRESS: '0xf70da97812CB96acDF810712Aa562db8dfA3dbEF',
} as const;
```

The `BRIDGE_ADDRESS` ie `0xf70da97812CB96acDF810712Aa562db8dfA3dbEF` is pointing to a relay solver, see docs

The Relay solver uses intent with order IDs to route deposits,if a transfer arrives at the solver address without a recognized order ID, it wouldn't be matched to any intent.

Location of Affected Code

```
File: hyperliquid-cross-chain.ts
Function: depositUsdcFromEthereumToHyperLiquidWithPrivy()
Config: ETHEREUM_USDC_CONFIG.BRIDGE_ADDRESS
```

Impact

Every invocation of this function where `receiverAddress` is `BRIDGE_ADDRESS` will transfer the user's USDC to a Relay solver contract on Ethereum mainnet with no associated intent. The solver has no context to route the funds and will not act on them.

Recommendation

If the intention was to use a relay solver, then first fetch a quote from the relay, as the solver must have an intent. https://docs.relay.link/how-it-works/the-relay-solver

Team Response

Fixed.

[L-06] WSOL Balance Overwritten by SPL Token Scan in `getAllSolanaTokenBalances()`

Severity

Low Risk

Description

The function `getAllSolanaTokenBalances()` first stores the native SOL balance under `SOLANA_TOKEN_ADDRESS` (which equals the WSOL mint address), then iterates over all SPL token accounts, which may include a WSOL token account

```
// Step 1: Store native SOL balance under WSOL address
balances[SOLANA_TOKEN_ADDRESS] = solBalance / 1e9;

// Step 2: Iterate SPL tokens —if WSOL account exists, overwrites step 1
for (const account of tokenAccounts.value) {
  const mintAddress = parsedInfo.mint;
  balances[mintAddress] = tokenAmount.uiAmount; // overwrites if mint === WSOL address
}
```

If the wallet holds both native SOL and WSOL, the WSOL SPL token balance will silently overwrite the native SOL balance in the map, losing the native SOL balance entirely.

Location of Affected Code

```
File: balance.service.ts
Function: getAllSolanaTokenBalances()
```

Impact

The returned balance map will contain incorrect data for wallets holding both native SOL and WSOL: – Native SOL balance is lost, replaced by the WSOL SPL token balance – The two assets are indistinguishable in the returned map – Any consumer relying on the native SOL balance will read the WSOL balance instead, or vice versa

Recommendation

Use distinct keys for native SOL and WSOL

```
// Store native SOL under its own canonical address
balances[NATIVE_SOL_ADDRESS] = solBalance / 1e9;

// SPL token scan stores WSOL under its own mint address separately
for (const account of tokenAccounts.value) {
  const mintAddress = parsedInfo.mint;
  balances[mintAddress] = tokenAmount.uiAmount;

}
```

This preserves both values independently and makes the distinction explicit to callers.

Team Response

Fixed.

[L-07] `symbol` Passed as String Where Hyperliquid API Requires Numeric Coin Index in `placeOrderWithPrivy()`

Severity

Low Risk

Description

The order request when placing an order via `placeOrderWithPrivy()` is constructed with `params.symbol` assigned to the `a` field

```
const orderReq: Record<string, unknown> = {
  a: params.symbol,
  // code
};
```

The Hyperliquid API requires the `a` field to be a **numeric coin index**, not a string symbol. The interface `HyperliquidPlaceOrdersParams` correctly uses `coin: number` for this field, but `HyperliquidPlaceOrderParams` uses `symbol: string`, creating an inconsistency between the two interfaces that results in an incorrect API payload.

Submitting a string value for `a` will cause the Hyperliquid API to reject the order with a validation error.

https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/exchange-endpoint

Location of Affected Code

File: hyperliquid.service.ts
Function: placeOrderWithPrivy()

Impact

Every order placed through `placeOrderWithPrivy()` will be rejected by the HyperliquidAPI. The `a` field is a required field in every order request

Recommendations

Option A Accept a numeric coin index in `HyperliquidPlaceOrderParams`

```
interface HyperliquidPlaceOrderParams {
  coin: number; // numeric index, consistent with HyperliquidPlaceOrdersParams
  // code
}
```

Option B, like many SDKs, consider adding a helper that converts the symbol to a number.

```
const coinIndex = await HyperliquidMetaService.getCoinIndex(params.symbol);
a: coinIndex,
```

Team Response

Fixed.

[L-08] USDC Balance Checked Using Native SOL Method for SPL Token in `depositFromSolanaWithFeeSponsorship()`

Severity

Low Risk

Description

The function `depositFromSolanaWithFeeSponsorship()` checks the user's balance using `connection.getBalance()` for both SOL and USDC

```
const shouldCheckUserBalance = currency === TokenAddress.SOLANA_USDC ||
  currency === '11111111111111111111111111111111';

const [feeSponsorBalance, userBalance, { blockhash }] = await Promise.all([
  connection.getBalance(executingWallet.publicKey),
  shouldCheckUserBalance ? connection.getBalance(userPubkey) : Promise.resolve(0),
  connection.getLatestBlockhash(),
]);
```

If currency is either `SOLANA_USDC` or `native`, then `shouldCheckUserBalance` is true.

`connection.getBalance()` is a **native SOL balance method** which returns the lamport balance of an account. USDC on Solana is an **SPL token**, held in a separate associated token account and calling `getBalance()` on a user's public key for a USDC check will return their SOL balance, not their USDC balance.

The correct method for SPL token balances is

```
connection.getTokenAccountBalance(associatedTokenAddress)
```

Location of Affected Code

```
File: hyperliquid-deposit.service.ts
Function: depositFromSolanaWithFeeSponsorship()
```

Impact

The user balance check for USDC will always read the user's SOL balance instead. Since the user balance is only used for logging, we rank this as a low for now

Recommendation

Use the correct SPL token balance check for USDC, and keep the native SOL check only for native tokens, however, since the user balance is not being used anywhere, only logging, we might as well get rid of it.

Team Response

Fixed.

[I-01] Consider Hardening Referrals Due To Farming via Unrestricted Account Creation

Severity

Informational Risk

Description

The platform implements a referral system where users receive "Crush Points" based on the number of referred accounts. However, there are no observable restrictions preventing a single actor from creating multiple accounts using their own referral code. The application shows the referees counted in real time when a new account is referred.

In the absence of anti-abuse or Sybil-resistance mechanisms, an attacker can repeatedly create fake or low-effort accounts to artificially inflate referral counts and accumulate points, or increase the referrals breaking the internal system or leaderboards, without meaningful economic cost or participation.

This enables referral farming and undermines the integrity of the points and leaderboard system.

Vulnerable Scenario: The following steps help understand the issue: 1. A user generates a referral code. 2. The user creates multiple new accounts using the same referral code. 3. Each account is counted as a valid referral without strong verification or activity requirements. 4. The attacker accumulates referral points, or referral counts, disproportionate to real user adoption.

Affected Files

Observed at the application/business-logic level

Relevant components: - Referral system - Account creation flow - Points/rewards calculation logic

Impact

Referral farming allows a single actor to manipulate platform incentives, potentially leading to unfair rewards, leaderboard distortion, and economic abuse. If points later translate into financial rewards, tokens, fee discounts, or governance rights, this issue may result in direct financial and governance impact.

Recommendations

Introduce anti-Sybil and anti-abuse controls for referrals, such as: - Minimum on-chain activity or trading volume before referral rewards are granted - Delayed or vested referral point issuance - Self-referral detection (shared wallet funding sources, device/IP heuristics) - Rate-limiting account creation and referral attribution - Wallet age or transaction history requirements

Team Response

Acknowledged.

[I-02] Misleading Error Message for Minimum Deposit Validation in `depositUsdcFromArbitrumToHyperLiquidWithPrivy()`

Severity

Informational Risk

Description

The function `depositUsdcFromArbitrumToHyperLiquidWithPrivy()` enforces a minimum deposit amount before transferring USDC to the Hyperliquid Bridge2 contract

```
  if (amountNum < 5) {
    throw new Error(
      'Amount must be greater than 5 USDC when transferring to bridge2',
    );
}
```

The condition correctly rejects amounts strictly less than 5 USDC (`amountNum < 5`), meaning a deposit of exactly 5 USDC is valid and will proceed. However, the error message states must be greater than 5 USDC, which implies that 5 USDC itself is also rejected. This is inconsistent with the actual validation logic and contradicts the Hyperliquid documentation, which states the minimum deposit is $5 USDC inclusive.

Location of Affected Code

File: hyperliquid-cross-chain.ts
Function: depositUsdcFromArbitrumToHyperLiquidWithPrivy()

Impact

While this does not affect the correctness of the validation logic or result in any loss of funds, the misleading error message could cause confusion. A user who receives this error when attempting a 5 USDC deposit may incorrectly believe the transaction requires more than 5 USDC

Recommendation

Update the error message to accurately reflect the boundary condition:

```
 if (amountNum < 5) {
  throw new Error(
    'Amount must be at least 5 USDC when transferring to bridge2',
  );
}
```

Team Response

Fixed.

[I-03] Redundant Branch in Secret Pattern Sanitizer

Severity

Informational Risk

Description

The function `sanitizeOutput()` iterates over secret patterns and applies replacements, but the if/else branch on the replacement type is entirely redundant, as both branches execute identical logic and have no effect on behavior, introducing unnecessary complexity and suggesting a likely oversight or unfinished differentiation between cases.

```
if (typeof replacement === 'function') {
  sanitized = sanitized.replace(pattern, replacement);
} else {
  sanitized = sanitized.replace(pattern, replacement);
}
```

Regardless of whether `replacement` is a function or something else, the type check adds no difference and suggests either a copy-paste error or an incomplete implementation where the two branches were intended to do different things.

Location of Affected Code

```
File: /src/middleware/sanitizer.ts
Function: sanitizeOutput()
```

Impact

The type check implies the two cases are handled differently, which is false according to this implementation.

Recommendation

If the original intent was to handle replacements differently depending on the type, the logic should be implemented

Team Response

Fixed.

[I-04] Duplicated Code Instances on `src/types/okx.ts`

Severity

Informational Risk

Description

When defining Types and interfaces for OKX DEX aggregator integration, some of the interfaces are being defined twice, which is just redundant:

```
// Balance API types
export interface OKXTokenBalanceRequest {
```

```
// Request for getting all token balances across chains
export interface OKXAllTokenBalancesRequest {
```

```
export interface OKXTransactionsByAddressRequest {
```

Location of Affected Code

File: src/types/okx.ts

Recommendation

Remove one of the instances for every duplicated interface.

Team Response

Fixed.

[I-05] `console.log()` Left in Production Code in Multiple Instances

Severity

Informational Risk

Description

A debug `console.log()` statement has been left in several functions. These logs should be removed:

```
console.log(builderConfig, 'builder config');
```

This logs the full `builderConfig` object to stdout on every order placement request.

Location of Affected Code

File: order-handlers.ts

```
function handlePlaceOrders() {
  // code
  console.log(builderConfig, 'builder config');
  console.log('error', error);
  // code
}
```

other instances

File: src/middleware/auth.ts

```
async function checkApiKeyWithCache(
  apiKey: string,
  allowedTypes: ApiKeyType[],
): Promise<boolean> {

  console.log('result', result);
  console.log('allowedTypes', allowedTypes);
}
```

File: /src/middleware/validation.ts

```
export function validateBody<T extends z.ZodType>(schema: T) {
    //code
    if (!result.success) {
      console.log('result.error.errors', result.error.errors);
    }
    //code
}
```

Recommendation

Remove the statement and replace it with the structured logger if it's really needed.

Team Response

Fixed.

[I-06] Redundant `SOLANA_TOKEN_ADDRESS` Constant Duplicates `WRAPPED_SOL_ADDRESS`

Severity

Informational Risk

Description

Two constants are defined with identical values:

```
const WRAPPED_SOL_ADDRESS = 'So11111111111111111111111111111111111111112';
const SOLANA_TOKEN_ADDRESS = 'So11111111111111111111111111111111111111112'; // identical
```

Location of Affected Code

File: balance.service.ts

Recommendation

Remove `SOLANA_TOKEN_ADDRESS` entirely and use `WRAPPED_SOL_ADDRESS` consistently throughout the file.

Team Response

Fixed.

[I-07] Multiple token Accounts for the same mint can still overwrite balance

Severity

Informational Risk

Description

A user may own arbitrarily many token accounts belonging to the same mint which means a user's balance for one token can be split across several accounts instead of sitting in a single place.

According to solana wallet integration guide:

"For each token mint, the wallet could have multiple token accounts: the associated token account and/or other ancillary token accounts."

```
// Process each token account
    for (const account of tokenAccounts.value) {
      try {
```

```
            const parsedInfo = account.account.data.parsed.info;
            const mintAddress = parsedInfo.mint;
            const tokenAmount = parsedInfo.tokenAmount;

            if (tokenAmount.uiAmount && tokenAmount.uiAmount > 0) {
              balances[mintAddress] = tokenAmount.uiAmount;//@audit increment balance
↪ incase wallet has multiple accounts:
              //https://www.solana-program.com/docs/token#finding-all-token-accounts-
↪ for-a-specific-mint
            }
          } catch (error) {
            logError(error, {
              context: 'Error processing Solana token account',
              walletAddress,
            });
            // Continue with next token account
          }
        }
      }
```

The function `getSolanaTokenBalance` only takes the first account that matches, meaning if multiple exits, they will be ignored.

```
// Get the balance from the first matching token account
const tokenAccount = tokenAccounts.value[0];
const tokenAmount = tokenAccount.account.data.parsed.info.tokenAmount; //@audit why only
↪ the first token account though?
```

Location of Affected Code

```
File: balance.service.ts
Function: getAllSolanaTokenBalances()
Function: getSolanaTokenBalance()
```

Impact

The balance will be overwritten if multiple token accounts exists for the same token returning only the last one. In getSolanaTokenBalance(), only the first account is being matched

Recommendation

Increment the value instead of directly assigning a new value

Reference

Rareskills spl-token wallet-integration-guide

Team Response

Fixed.

shieldify

Thank you!